

EIP v1.1 - Coder Agent Instructions - Step 1 Foundation Pipeline

EIP v1.1 – Coder Agent Instructions – Step 1 Foundation Pipeline

Purpose

Implement the first production-ready stage of the Executive Intelligence Pipeline (EIP) in a new SharePoint location:

- Site: Petersplace
- Library: Shared Documents
- Root folder: EIP version 1.1

This step must deliver a fully useful working pipeline, not a proof of concept.

The implementation must be:

- VTT-first
- topic-first
- people-aware
- retrieval-first
- configuration-driven
- traceable
- repeatable
- suitable for later trend, comparison and executive-intelligence layers

The current pipeline version 1.7.x already acquires raw Teams transcript files through PowerShell scripts and Microsoft Graph GET operations. EIP version 1.1 must build on that acquisition model and add a reliable processing, classification, artefact and retrieval foundation.

SharePoint is the storage and artefact target. It is not the transcript acquisition source.

1. Version Scope

1.1 Version 1.1 delivers now

EIP v1.1 must implement:

- Microsoft Graph / PowerShell VTT acquisition interface
- VTT ingestion
- transcript validation
- transcript normalisation

- timestamp preservation in metadata
- speaker and turn resolution
- internal transcript model generation
- semantic segmentation
- summary generation
- topic extraction
- deterministic topic identity
- taxonomy classification
- classification traceability
- people-aware signal extraction
- topic record generation
- retrieval artefact generation
- recommendation artefact generation
- append-only master log generation
- SharePoint artefact persistence
- validation and error reporting

1.2 Version 2.x delivers later

Do not implement these in v1.1, but prepare the artefacts so they can be added later:

- topic history analysis
- trend detection
- drift analysis
- stalled-work detection
- deviation-from-plan detection
- comparison engine
- longitudinal topic state

1.3 Version 3.x delivers later

Do not implement these in v1.1:

- executive intelligence layer
- strategic signal generation
- state-of-business generation
- advanced Copilot retrieval support
- executive dashboard layer
- executive reporting layer

1.4 Scope control rule

Do not implement Version 2.x or Version 3.x capabilities in this phase.

Only prepare v1.1 artefacts so those capabilities can be built later without a major redesign.

2. Scope Boundary

This implementation phase is the ingestion, transcript normalisation, semantic segmentation, summary generation, topic classification and artefact-generation foundation.

It must:

1. Acquire or receive raw Microsoft Teams VTT transcript files through the existing PowerShell and Microsoft Graph GET acquisition flow.
2. Capture associated meeting metadata where available.
3. Validate the raw VTT input.
4. Normalise and clean transcript content.
5. Identify speakers and speaking turns.
6. Preserve timestamp references in metadata.
7. Generate a structured internal transcript model.
8. Generate semantic transcript segments.
9. Generate an executive meeting summary as an internal processing artefact.
10. Extract and classify topic-level records.
11. Resolve people and ownership metadata.
12. Generate structured meeting, topic, master-log, retrieval and recommendation artefacts.
13. Write all artefacts into the new SharePoint folder structure.
14. Produce append-only master-log entries.
15. Be fully configuration-driven.
16. Operate without manual rewriting of transcript content.

This phase must not:

- perform full trend analysis
- perform full stalled-work analysis
- perform full deviation analysis
- generate executive dashboards
- generate final executive reports
- automatically mutate taxonomy
- automatically mutate mapping rules
- automatically mutate the people registry

Preparation for those later capabilities must be built into the produced artefacts from day one.

3. Non-Negotiable Design Principles

3.1 Configuration first

No taxonomy values may be hardcoded.

No domains may be hardcoded.

No classifications may be hardcoded.

No mapping rules may be hardcoded.

No people mappings may be hardcoded outside the approved people configuration.

No topic identity rules may be hardcoded.

No retrieval rules may be hardcoded.

Everything must come from configuration files.

3.2 VTT first

The primary source input is the raw Teams VTT transcript.

Generated summaries are internal artefacts.

The generated summary must never be treated as the primary source of truth.

3.3 Source-of-truth hierarchy

The system must prefer data sources in the following order:

1. Raw VTT transcript
2. Transcript model
3. Semantic segments
4. Generated summary
5. Topic artefacts
6. Retrieval artefacts

Generated artefacts must never supersede source transcript evidence.

When conflicts exist, upstream artefacts take precedence over downstream artefacts.

The raw VTT and transcript model must remain available for audit, validation and reconstruction.

3.4 Topic first

The topic is the primary intelligence object.

Never store executive intelligence only at meeting level.

Everything must ultimately become topic-centric records.

Meeting-level artefacts may exist, but they are supporting artefacts.

3.5 Deterministic topic identity

Topic IDs must be deterministic and consistent across meetings whenever the organisation is discussing the same underlying topic.

Topic IDs must be based on taxonomy identity, operational area and topic meaning, not random GUIDs.

Examples:

- ```
1 PRODUCT.SUPERPEN.MVP
2 PRODUCT.SUPERPEN.THERMAL
3 DELIVERY.NPI.STAGE3
4 WORKMETHODOLOGY.AUTOMATION
```

Plain Text ▾

If a stable taxonomy topic cannot be assigned, create a provisional topic ID using the configured provisional-topic rules and mark it as unresolved.

Do not silently create new taxonomy topics.

## 3.6 People aware

Every extracted signal must preserve, where available:

- owner
- mentioned people
- participants
- speakers
- organiser
- source meeting context

People become metadata.

Unknown people must not be discarded.

## 3.7 Retrieval first

The pipeline must generate artefacts that are easy for Copilot, Microsoft Search and future semantic retrieval layers to use.

Do not optimise only for storage.

Each topic artefact must include stable identifiers, readable summaries, keywords, aliases, people references, source references and classification metadata.

All retrieval artefacts must be designed for grounding and machine retrieval rather than only human reading.

Every topic artefact must include:

- executiveSummary
- semanticSummary
- keywords
- aliases
- people
- products
- projects
- operationalAreas
- sourceMeetings
- sourceTranscript
- sourceSegments
- sourceTurns
- classification metadata
- version metadata

## 3.8 Traceability first

Every extracted field should preserve the best available source reference.

Where possible, extracted values must include:

JSON ▾

```
1 {
2 "value": "string",
3 "source": "rawVtt | transcriptModel | semanticSegment | generatedSummary",
4 "sourceSection": "string or null",
5 "sourceSegmentId": "string or null",
6 "sourceTurnId": "string or null",
7 "speaker": "string or null",
8 "timestampRange": "string or null",
9 "confidence": 0
10 }
11
```

Do not output unsupported conclusions as if they were facts.

When evidence is weak, preserve the signal but lower confidence.

## 3.9 Append only

Master log records can never be modified.

Only new records may be appended.

History is sacred.

## 3.10 Taxonomy governance

Taxonomy modifications are never performed automatically.

The system may produce:

- taxonomy recommendations
- mapping recommendations
- keyword recommendations
- alias recommendations
- people-resolution recommendations

Human approval is always required before taxonomy, mapping or people-registry changes become active.

Recommendations must be written as separate artefacts and must not mutate active configuration files.

---

# 4. Target Folder Layout

Create the following structure under:

Plain Text ▾

```
1 Shared Documents/EIP version 1.1/
2
```

Required folder layout:

Plain Text ▾

```
1 EIP version 1.1/
2
3 config/
4 pipeline.json
5 taxonomy.json
6 people.json
7 mapping_rules.json
8 extraction_rules.json
9 validation_rules.json
10 retrieval_rules.json
11 topic_identity_rules.json
12
13 input/
14 vtt/
15 metadata/
16
17 processing/
18 normalised-transcripts/
19 transcript-models/
20 semantic-segments/
21 generated-summaries/
22
23 artifacts/
24 meetings/
25 topics/
26 retrieval/
27 recommendations/
28 logs/
29
30 master-log/
31 master_log.jsonl
32
33 errors/
34
35 archive/
36 raw-vtt/
37 processed-vtt/
38
```

## 5. Required Input Contract

Pipeline input is a raw Microsoft Teams VTT transcript file acquired through the existing PowerShell and Microsoft Graph GET transcript acquisition process.

The pipeline must not expect a manually prepared JSON meeting-summary input.

Input format:

Plain Text ▾

```
1 WEBVTT
2
3 00:00:00.000 --> 00:00:05.000
4 Speaker Name: Transcript text
5
6 00:00:05.000 --> 00:00:10.000
```

```
7 Speaker Name: Transcript text
8
```

The current pipeline version 1.7.x already acquires raw VTT files through Microsoft Graph APIs and PowerShell acquisition scripts.

EIP version 1.1 must consume those acquired VTT files as its source input.

Meeting metadata must be collected from available acquisition context, Microsoft Graph context and transcript file context where available.

Metadata may include:

- meeting id
- meeting title
- meeting date
- organiser
- participants
- transcript id
- transcript file name
- source acquisition timestamp
- source Graph endpoint or identifier

The first processing stage must perform transcript normalisation and cleaning before summary generation.

The generated summary becomes an internal processing artefact and must not be treated as the primary input.

The pipeline must reject invalid or empty VTT inputs and write a validation error artefact.

---

## 6. Required Processing Flow

```
1 Microsoft Graph GET
2 ↓
3 PowerShell Transcript Acquisition
4 ↓
5 Raw VTT
6 ↓
7 Transcript Validation
8 ↓
9 Transcript Normalisation
10 ↓
11 Speaker and Turn Resolution
12 ↓
13 Internal Transcript Model
14 ↓
15 Semantic Segmentation
16 ↓
17 Summary Generation
18 ↓
19 Topic Extraction
20 ↓
21 Taxonomy Classification
22 ↓
23 People Resolution
24 ↓
25 Signal Extraction
26 ↓
```

Plain Text ▾



```
27 Topic Record Generation
28 ↓
29 Retrieval Artefact Generation
30 ↓
31 Recommendation Artefact Generation
32 ↓
33 Append-only Master Log
34 ↓
35 SharePoint Artefact Storage
36
```

---

## 7. Implementation Steps

### Step 1A – Configuration Loader

Build a configuration service.

Load:

- pipeline.json
- mapping\_rules.json
- taxonomy.json
- people.json
- extraction\_rules.json
- validation\_rules.json
- retrieval\_rules.json
- topic\_identity\_rules.json

Validate:

- schema
- required properties
- version fields
- enabled domains
- enabled classifications
- mapping-rule structure
- people-registry structure
- retrieval-rule structure
- topic-identity-rule structure

Fail fast on invalid configuration.

Generate:

- ConfigurationContext object

This object becomes the only source of truth during execution.

### Step 1B – Transcript Acquisition Interface

Use the existing PowerShell and Microsoft Graph GET acquisition flow as the source mechanism for raw

transcript files.

The coder must not replace this with manual SharePoint upload logic.

Responsibilities:

- accept acquired VTT files from the PowerShell acquisition workflow
- preserve source file name
- preserve acquisition timestamp
- preserve Graph-derived metadata where available
- place acquired VTT files into the configured input or archive location
- pass raw VTT and metadata to the processing pipeline

Output:

- RawTranscriptInput object

Minimum fields:

```
1 {
2 "sourceType": "MicrosoftGraphTranscript",
3 "fileName": "string",
4 "rawVttText": "string",
5 "metadata": {},
6 "acquiredAt": "ISO datetime"
7 }
8
```

JSON ▾

## Step 1C – Transcript Validation

Validate the raw VTT input before any semantic processing.

Validation must check:

- file exists
- file is not empty
- file appears to be VTT format
- transcript contains timestamp blocks
- transcript contains spoken text
- transcript can be parsed into turns

Invalid files must be moved or copied to:

```
1 errors/
2
```

Plain Text ▾

Generate an error report including:

- file name
- validation failure reason
- timestamp
- pipeline version

## Step 1D – Transcript Normalisation

Input:

- Raw VTT transcript

Responsibilities:

- remove VTT headers where appropriate
- remove timestamps from analytical text
- preserve original timestamp references in metadata for traceability and source reconstruction
- remove transcript formatting noise
- preserve speaker names where available
- identify speaker turns
- merge fragmented turns where appropriate
- remove duplicate caption fragments where safe
- preserve source offsets or timestamp references for traceability
- produce clean transcript text
- produce structured transcript turns

Output:

- NormalisedTranscript object

Minimum structure:

```
1 {
2 "meetingId": "string or null",
3 "meetingTitle": "string or null",
4 "date": "ISO date or null",
5 "participants": [],
6 "turns": [
7 {
8 "turnId": "string",
9 "speaker": "string or unknown",
10 "startTime": "string or null",
11 "endTime": "string or null",
12 "text": "string",
13 "sourceOffset": "string or null"
14 }
15]
16 }
17
```

JSON ▾

## Step 1E – Speaker and People Resolution

Resolve speakers, mentioned people, owners and participants.

Use people.json as the canonical people registry.

Store both:

- display name
- canonical id

Unknown people must remain in the artefact but be flagged as unresolved.

Do not discard unresolved people references.

All people-resolution outputs must include PEOPLE\_VERSION.

## Step 1F – Semantic Segmentation

Convert the normalised transcript into semantic segments.

Segmentation should identify coherent discussion blocks before summary generation.

Use deterministic rules first where possible:

1. agenda markers
2. topic shifts
3. speaker transitions
4. repeated subject references
5. decision or action language
6. explicit project or product names

Use LLM assistance only where deterministic segmentation is insufficient.

Output:

- SemanticSegment objects

Minimum fields:

```
1 {
2 "segmentId": "string",
3 "title": "string",
4 "startTurnId": "string",
5 "endTurnId": "string",
6 "speakers": [],
7 "text": "string",
8 "candidateTopics": [],
9 "confidence": 0
10 }
11
```

JSON ▾

## Step 1G – Summary Generation

Generate an executive meeting summary from the normalised transcript and semantic segments.

The summary is an internal processing artefact.

It must support downstream extraction, not replace source evidence.

The generated summary should include:

- concise executive summary
- main topics
- decisions
- actions
- risks
- issues
- implications
- alignment or drift signals

- next direction

Store the generated summary in:

Plain Text ▾

```
1 processing/generated-summaries/
2
```

## Step 1H – Topic Extraction

Convert semantic segments and the generated summary into topic-level units.

Processing order:

1. explicit topic references from transcript
2. semantic segment titles
3. repeated project/product/process references
4. generated summary topic headings
5. LLM-assisted grouping where needed

Output:

- Topic objects

Minimum fields:

JSON ▾

```
1 {
2 "topicId": "stable deterministic id",
3 "topicTitle": "string",
4 "content": "string",
5 "sourceSegmentIds": [],
6 "sourceTurnIds": [],
7 "people": [],
8 "confidence": 0
9 }
10
```

Prevent duplicate topic creation.

Merge overlapping or near-duplicate topics.

Topic IDs must be stable where possible and based on taxonomy and topic identity, not random GUIDs.

## Step 1I – Taxonomy Classification

For each topic assign:

- Domain
- Classification
- Topic
- ContextType

Use:

1. deterministic mapping rules

2. taxonomy aliases and keywords
3. semantic similarity
4. LLM fallback only when required

Every assignment must include:

- confidence
- classification source
- rule id used where applicable
- taxonomy version
- mapping version
- matched keywords
- matched aliases

Classification output must use this structure where possible:

JSON ▾

```
1 {
2 "domain": "string",
3 "classification": "string",
4 "topic": "string",
5 "contextType": "string",
6 "matchedKeywords": [],
7 "matchedAliases": [],
8 "ruleId": "string or null",
9 "classificationSource": "deterministic | semantic | llmFallback",
10 "confidence": 0,
11 "taxonomyVersion": "string",
12 "mappingVersion": "string"
13 }
14
```

## Step 1J – Signal Extraction

For every topic extract:

- Signals
- Decisions
- Actions
- Action owners
- Risks
- Issues
- Implications
- Alignment
- Direction

Required direction states:

- Improving
- Stable
- Declining
- Unclear

Required alignment states:

- Aligned
- Drift
- Unknown

Each extracted signal must include source reference where possible:

- source segment id
- source turn id
- speaker
- timestamp range where available
- confidence

## Step 1K – Confidence Engine

Generate topic-level confidence score.

Factors:

- transcript quality
- speaker resolution quality
- segmentation quality
- extraction completeness
- classification certainty
- owner resolution
- mapping certainty
- source traceability

Output:

- 0–100

## Step 1L – Topic Record Generator

Generate one record per topic.

Mandatory fields:

- RECORD\_ID
- MEETING\_ID
- MEETING\_TITLE
- DATE
- PARTICIPANTS
- ORGANISER
- DOMAIN
- TOPIC
- CLASSIFICATION
- CONTEXT\_TYPE
- SIGNALS
- DECISIONS
- ACTIONS
- ACTION\_OWNERS

- RISKS
- ISSUES
- IMPLICATIONS
- ALIGNMENT
- DIRECTION
- CONFIDENCE\_SCORE
- SOURCE\_TRANSCRIPT
- SOURCE\_SEGMENTS
- SOURCE\_TURNS
- PIPELINE\_VERSION
- TAXONOMY\_VERSION
- MAPPING\_VERSION
- PEOPLE\_VERSION
- RETRIEVAL\_VERSION
- TIMESTAMP

Master-log records must be append-only.

## Step 1M – Retrieval Artefact Generator

Generate retrieval-focused artefacts from day one.

For each topic, generate retrieval metadata:

- stable topic id
- topic title
- executive topic summary
- semantic summary
- keywords
- aliases
- related people
- related products
- related projects
- related operational area
- source meeting
- source date
- source transcript
- source segment ids
- source turn ids
- confidence
- pipeline version
- taxonomy version
- mapping version
- people version
- retrieval version

Create or update retrieval files under:



Plain Text ▾

```
1 artifacts/retrieval/
2
```

Minimum retrieval artefacts:

- meeting\_index.json
- topic\_index.json
- people\_index.json
- keyword\_index.json

Retrieval artefacts are not dashboards.

They are structured indexes to make future semantic search and Copilot retrieval more reliable.

Retrieval artefacts must be machine-readable, compact and consistent.

## Step 1N – Recommendation Artefact Generator

Generate recommendations where the system detects weak or missing configuration coverage.

This may include:

- unknown topic candidates
- weak classification confidence
- suggested taxonomy aliases
- suggested keyword additions
- suggested mapping-rule additions
- unresolved people references

Write recommendations to:

Plain Text ▾

```
1 artifacts/recommendations/
2
```

Do not automatically update active configuration files.

## Step 1O – SharePoint Persistence Layer

Write outputs to SharePoint under:

Plain Text ▾

```
1 Shared Documents/EIP version 1.1
2
```

Meeting artefacts:

Plain Text ▾

```
1 artifacts/meetings/YYYY/MM/
2
```

Topic artefacts:

```
1 artifacts/topics/
2
```

Plain Text ▾

Retrieval artefacts:

```
1 artifacts/retrieval/
2
```

Plain Text ▾

Recommendation artefacts:

```
1 artifacts/recommendations/
2
```

Plain Text ▾

Processing artefacts:

```
1 processing/
2
```

Plain Text ▾

Master log:

```
1 master-log/master_log.jsonl
2
```

Plain Text ▾

Each topic record must be appended as a new line.

Never rewrite historic master-log records.

## Step 1P – Validation Layer

Validate every produced artefact.

Required checks:

- transcript model exists
- generated summary exists

- topic exists
- deterministic topic id exists
- domain exists
- classification exists
- context type exists
- confidence exists
- record id exists
- source reference exists
- pipeline version exists
- taxonomy version exists
- mapping version exists
- people version exists
- retrieval version exists where relevant

Invalid records must be written to:

```
1 errors/
2
```

Plain Text ▾

Generate an error report.

Continue processing remaining valid records where safe.

---

## 8. Required Artefact Contracts

### 8.1 Meeting artefact

Each processed meeting must generate one meeting artefact.

Minimum fields:

```
1 {
2 "meetingId": "string or null",
3 "meetingTitle": "string or null",
4 "date": "ISO date or null",
5 "organiser": "string or null",
6 "participants": [],
7 "sourceTranscript": "string",
8 "generatedSummary": "string",
9 "semanticSegmentIds": [],
10 "topicIds": [],
11 "pipelineVersion": "string",
12 "taxonomyVersion": "string",
13 "mappingVersion": "string",
14 "peopleVersion": "string",
15 "retrievalVersion": "string",
16 "createdAt": "ISO datetime"
17 }
18
```

JSON ▾

## 8.2 Topic artefact

Each topic must generate one topic artefact.

Minimum fields:

JSON ▾

```
1 {
2 "topicId": "string",
3 "topicTitle": "string",
4 "domain": "string",
5 "classification": "string",
6 "contextType": "string",
7 "executiveSummary": "string",
8 "semanticSummary": "string",
9 "signals": [],
10 "decisions": [],
11 "actions": [],
12 "risks": [],
13 "issues": [],
14 "implications": [],
15 "alignment": "Aligned | Drift | Unknown",
16 "direction": "Improving | Stable | Declining | Unclear",
17 "people": [],
18 "keywords": [],
19 "aliases": [],
20 "sourceTranscript": "string",
21 "sourceSegmentIds": [],
22 "sourceTurnIds": [],
23 "confidenceScore": 0,
24 "versions": {
25 "pipeline": "string",
26 "taxonomy": "string",
27 "mapping": "string",
28 "people": "string",
29 "retrieval": "string"
30 }
31 }
32
```

## 8.3 Master log record

The master log must be JSONL.

Each line is one topic record.

Minimum fields:

JSON ▾

```
1 {
2 "RECORD_ID": "string",
3 "MEETING_ID": "string or null",
4 "MEETING_TITLE": "string or null",
5 "DATE": "ISO date or null",
6 "PARTICIPANTS": [],
7 "ORGANISER": "string or null",
8 "DOMAIN": "string",
9 "TOPIC": "string",
10 "CLASSIFICATION": "string",
11 "CONTEXT_TYPE": "string",
12 "SIGNALS": [],
13 "DECISIONS": [],

```

```

14 "ACTIONS": [],
15 "ACTION_OWNERS": [],
16 "RISKS": [],
17 "ISSUES": [],
18 "IMPLICATIONS": [],
19 "ALIGNMENT": "Aligned | Drift | Unknown",
20 "DIRECTION": "Improving | Stable | Declining | Unclear",
21 "CONFIDENCE_SCORE": 0,
22 "SOURCE_TRANSCRIPT": "string",
23 "SOURCE_SEGMENTS": [],
24 "SOURCE_TURNS": [],
25 "PIPELINE_VERSION": "string",
26 "TAXONOMY_VERSION": "string",
27 "MAPPING_VERSION": "string",
28 "PEOPLE_VERSION": "string",
29 "RETRIEVAL_VERSION": "string",
30 "TIMESTAMP": "ISO datetime"
31 }
32

```

## 8.4 Retrieval indexes

Minimum retrieval indexes:

Plain Text ▾

```

1 artifacts/retrieval/meeting_index.json
2 artifacts/retrieval/topic_index.json
3 artifacts/retrieval/people_index.json
4 artifacts/retrieval/keyword_index.json
5

```

Indexes must be compact, consistent and machine-readable.

Do not bury the key retrieval fields inside long prose blocks.

# 9. Deliverables Required Before Phase Completion

The coder must demonstrate:

1. Configuration loading
2. Microsoft Graph / PowerShell VTT acquisition interface
3. VTT ingestion
4. Transcript validation
5. Transcript normalisation
6. Timestamp preservation in metadata
7. Speaker and turn resolution
8. People mapping
9. Semantic segmentation
10. Summary generation
11. Topic extraction
12. Deterministic topic identity
13. Taxonomy classification
14. Classification traceability

15. Signal extraction
16. Confidence scoring
17. Topic record generation
18. Retrieval artefact generation
19. Recommendation artefact generation
20. SharePoint persistence
21. Append-only master log generation
22. Error handling and validation reporting

Pipeline execution must be fully repeatable.

All outputs must be reconstructable solely from:

- configuration files
- raw VTT transcript
- acquisition metadata
- generated processing artefacts
- generated final artefacts

No hidden state permitted.

---

## 10. Acceptance Criteria

A raw Teams VTT transcript acquired through the Microsoft Graph / PowerShell acquisition flow can be processed by EIP version 1.1 and the system automatically produces:

- archived raw VTT transcript
- validated transcript input record
- normalised transcript
- internal transcript model
- semantic segments
- generated executive summary
- classified topic artefacts
- append-only master log entries
- retrieval artefacts
- recommendation artefacts where applicable
- traceable metadata
- confidence scores
- people-aware records
- validation report

with no manual rewriting of transcript content.

The output must preserve enough source references to reconstruct why each topic, classification, signal, action, risk or implication was created.

When this acceptance criterion is met, Step 1 is complete and the platform is ready for Phase 2: trend intelligence, comparison engine, stalled-work detection, deviation analysis, executive analytics and enhanced

## 11. Implementation Guardrails for the Coder Agent

The coder agent must follow these rules:

1. Do not simplify the pipeline back to summary-first.
2. Do not treat the generated summary as source truth.
3. Do not remove timestamps without preserving source timestamp references.
4. Do not create random topic IDs when deterministic topic identity is possible.
5. Do not mutate taxonomy, mapping rules or people registry automatically.
6. Do not store only human-readable summaries.
7. Do not omit retrieval artefacts.
8. Do not omit source references.
9. Do not rewrite the append-only master log.
10. Do not add Version 2.x or Version 3.x analytics in this phase.

The coding objective is a robust v1.1 foundation pipeline.

The success measure is not whether the output looks like a good meeting summary.

The success measure is whether later systems can reliably retrieve, compare, validate and reason over topic-level executive intelligence from raw meeting transcripts.